

AP06 - Estruturas de Repetição: Problemas Avançados

Iniciado: 14 Set em 9:07

Instruções do teste

AP06 - Roteiro Prático 6: Estruturas de Repetição - Problemas Avançados

Neste roteiro você irá retomar os padrões básicos de repetição estudados nas aulas anteriores e avançar para problemas que exigem maior controle sobre a execução do algoritmo.

Os exercícios começam com repetições simples e acesso a **strings por índice** e evoluem para **comparação entre posições**, **detecção de padrões**, **laços aninhados**, **substrings** e **contagem de operações**.

Regras desta lista:

- Utilize apenas os recursos estudados até o momento: variáveis, strings, condicionais, `while`, `for` e `range()`.
- Não utilize listas, funções, conjuntos ou dicionários.
- Quando o enunciado solicitar acesso por índice, utilize `texto[i]`.
- Não utilize funções prontas que resolvam diretamente o problema, como `find()`, `count()`, `sorted()` ou fatiamento, quando isso estiver indicado.
- Quando for solicitado contar operações, incremente o contador exatamente no ponto em que a operação analisada for executada.



Pergunta 1 0 pts

Exercício 1 Revisão: soma e contagem em um intervalo

Leia dois números inteiros positivos **A** e **B**, com `A < B`.

Percorra todos os números do intervalo de A até B, inclusive, e determine:

- quantos números são múltiplos de 3;
- a soma de todos os múltiplos de 3 encontrados;
- quantos números são pares.

Ao final, exiba os três resultados.

Carregar

Choose a File



Pergunta 2 0 pts

Exercício 2 Comparando duas strings por índice

Leia duas palavras de mesmo tamanho.

Compare as duas palavras posição por posição, utilizando índices, e determine quantas posições possuem caracteres diferentes.

Ao final, mostre também quais posições são diferentes.

Exemplo:

```
casa  
capa
```

As palavras diferem apenas na posição 2.

Importante: utilize `texto[i]`. Não percorra diretamente os caracteres com `for letra in texto`.

Carregar

Choose a File



Pergunta 3 0 pts

Exercício 3 Conversão manual de binário para decimal

Leia um número binário como **string**, contendo apenas os caracteres `0` e `1`.

Converta esse número para decimal sem utilizar funções prontas de conversão de base.

O programa deve percorrer a string por índice e construir o valor decimal progressivamente.

Dica: ao ler um novo dígito binário da esquerda para a direita, o valor acumulado pode ser atualizado por `decimal = decimal * 2 + digito`.

Exemplo: `1101` deve resultar em `13`.

Carregar

Choose a File



Pergunta 4 0 pts

Exercício 4 Palíndromo por comparação de posições

Leia uma palavra e determine se ela é um palíndromo.

Em vez de construir a palavra invertida, compare diretamente os caracteres das extremidades: primeiro com último, segundo com penúltimo e assim por diante.

Além do resultado, informe quantas comparações entre caracteres foram realizadas.

Não utilize: fatiamento, `reversed()` ou uma segunda string invertida.

Carregar

Choose a File



Pergunta 5 0 pts

Exercício 5 Todas as substrings de tamanho K

Leia uma string **texto** e um inteiro positivo **K**, com `K <= len(texto)`.

Mostre todas as substrings contíguas de tamanho K existentes no texto.

A substring deve ser construída manualmente, caractere por caractere, utilizando laços aninhados.

Ao final, informe quantos caracteres foram copiados para construir todas as substrings.

Não utilize: fatiamento como `texto[i:i+k]`.

Exemplo: para `texto = "casa"` e `K = 2`:

ca
as
sa

Carregar

Choose a File



Pergunta 6 0 pts

Exercício 6 Busca ingênua de substring

Leia duas strings: um **texto** e um **padrão**.

Encontre todas as posições do texto em que o padrão aparece.

Para cada possível posição inicial, compare manualmente os caracteres do padrão com os caracteres correspondentes do texto.

Ao final, mostre:

- a posição inicial de cada ocorrência encontrada;
- a quantidade total de ocorrências;
- a quantidade total de comparações entre caracteres.

Ocorrências sobrepostas devem ser consideradas.

Não utilize: `in`, `find()`, `index()` ou fatiamento.

Exemplo: em `ababa`, o padrão `aba` aparece nas posições 0 e 2.

Carregar

Choose a File



Pergunta 7 0 pts

Exercício 7 Maior bloco de caracteres iguais

Leia uma string não vazia e determine qual é o maior bloco de caracteres iguais e consecutivos.

Mostre:

- o caractere que forma o maior bloco;
- o tamanho desse bloco;
- a posição em que o bloco começa.

Se houver empate, considere o primeiro bloco encontrado.

Exemplo: em `aaabbccccdaa`, o maior bloco é `cccc`, de tamanho 4, iniciando na posição 5.

Carregar

Choose a File



Pergunta 8 0 pts

Exercício 8 Números primos e contagem de divisões

Leia um número inteiro positivo **N**.

Mostre todos os números primos entre 2 e N.

Para verificar se cada número é primo, teste possíveis divisores utilizando um laço interno.

O programa deve contar quantas operações de resto (`%`) foram realizadas durante toda a execução.

Assim que um divisor exato for encontrado, não é necessário continuar testando outros divisores daquele número.

Ao final, mostre:

- a quantidade de números primos encontrados;
- a quantidade total de operações de resto realizadas.

Carregar

Choose a File



Pergunta 9 0 pts

Exercício 9 Caractere mais frequente sem count()

Leia uma string não vazia e determine qual caractere aparece mais vezes.

Para cada posição da string, percorra novamente toda a string e conte quantas vezes aquele caractere aparece.

Mostre:

- o caractere mais frequente;
- a quantidade de ocorrências;
- a quantidade total de comparações entre caracteres realizadas pelo algoritmo.

Em caso de empate, considere o primeiro caractere que alcançar a maior frequência.

Não utilize: `count()`, listas, conjuntos ou dicionários.

Carregar

Choose a File



Pergunta 10 0 pts

Exercício 10 Substring repetida de tamanho K

Leia uma string **texto** e um inteiro positivo **K**.

Verifique se existe alguma substring de tamanho K que apareça em pelo menos duas posições iniciais diferentes.

As ocorrências podem se sobrepor.

Quando encontrar a primeira repetição, mostre:

- a substring repetida;
- as duas posições iniciais em que ela foi encontrada;
- a quantidade total de comparações entre caracteres realizadas até encontrar o resultado.

Se nenhuma substring de tamanho K se repetir, informe isso ao usuário.

Não utilize: fatiamento, `find()`, `count()`, listas, conjuntos ou dicionários.

Exemplo: em `banana`, com `K = 2`, a substring `an` aparece nas posições 1 e 3.

Carregar

[Choose a File](#)

Questões extras

As questões a seguir podem ser utilizadas por quem concluir o roteiro principal antes do fim da aula ou como continuidade da prática.



Pergunta 11 0 pts

Extra 1 Algoritmo de Euclides

Leia dois números inteiros positivos e calcule o máximo divisor comum (MDC) utilizando o algoritmo de Euclides.

A cada passo, calcule o resto da divisão entre os dois valores e substitua o par de números para continuar o processo.

Conte quantas operações de resto foram necessárias até obter o resultado.

Carregar

[Choose a File](#)

Pergunta 12 0 pts

Extra 2 Pares de dígitos com soma-alvo

Leia uma string formada apenas por dígitos e um valor inteiro alvo.

Analise todos os pares de posições (i, j) com $i < j$ e mostre os pares cujos dígitos somam o valor alvo.

Ao final, informe quantos pares foram testados.

Carregar



Pergunta 13 0 pts

Extra 3 Prefixos palíndromos

Leia uma palavra e mostre todos os seus prefixos que são palíndromos.

Para cada tamanho possível de prefixo, verifique o padrão comparando as posições simétricas daquele trecho.

Não utilize fatiamento.

Carregar

Salvo em 17:53